



Universidad de
los Andes
Colombia

Departamento de Ingeniería
Eléctrica y Electrónica

INTELIGENCIA DE NEGOCIOS - ISIS 3301

INFORME PROYECTO 1

Andres Mateo Chilito Avella *Código: 202214992*

a.chilitoa@uniandes.edu.co

Franklin Smith Fernandez Romero *Código: 202215103*

f.fernandezr@uniandes.edu.co

Juan Jose Cediel Borrero *Código: 202212212*

jj.cediel@uniandes.edu.co

Este documento presenta la solución del proyecto número uno fase dos de la materia
Inteligencia De Negocios.

Universidad de los Andes

Bogotá - Colombia

30 de marzo de 2025

1 Introducción

La creciente diseminación de noticias falsas en el ecosistema digital, particularmente en el dominio político, constituye un desafío complejo con profundas implicaciones para la gobernabilidad democrática, la confianza institucional y la cohesión social. La capacidad de discernir eficazmente entre información veraz y contenido manipulado es muy importante en el contexto actual. Este proyecto se enmarca en la búsqueda de soluciones tecnológicas basadas en Inteligencia de Negocios y Aprendizaje Automático para mitigar este fenómeno. La primera etapa del proyecto se concentró en la exploración de datos, la ingeniería de características y la construcción y evaluación comparativa de diversos modelos de clasificación de textos para la detección de noticias falsas en español, culminando con la selección de un buen modelo basado en métricas de desempeño, las cuales ya fueron mencionadas en un anterior documento. La presente Etapa 2 transita hacia la ingeniería y operacionalización de la solución analítica. El enfoque se centra en dos partes muy importantes: la automatización y replicabilidad del pipeline de machine learning, desde la preparación de datos hasta la persistencia y el reentrenamiento del modelo; Y por otra parte: el desarrollo de una aplicación interactiva y funcional que permita a un usuario final específico consumir los resultados del modelo de manera efectiva para apoyar sus decisiones y acciones. Este documento presenta de manera detallada la arquitectura de automatización implementada mediante pipelines y una API REST, el diseño y funcionalidades de la aplicación web desarrollada, la justificación de su valor en un contexto organizacional definido, los hallazgos de las pruebas de usuario y la gestión del trabajo colaborativo del equipo durante esta fase del proyecto.

2 Proceso de Automatización del Modelo y Acceso por API

En esta sección se detalla la implementación de un flujo de trabajo automatizado y la creación de la API para el modelo de detección de noticias falsas:

2.1 Automatización del Pipeline de Preparación y Evaluación

Para garantizar la consistencia, reproducibilidad y eficiencia del proceso, se implementó un pipeline utilizando la biblioteca Scikit-learn ('sklearn.pipeline.Pipeline'). Este pipeline encapsula la secuencia completa de pasos requeridos, desde el preprocesamiento del texto crudo hasta la clasificación final. La estructura del pipeline, definida en `pipeline/pipeline.py` (disponible en el repositorio del proyecto¹), es la siguiente:

1. **Preprocesamiento de Texto:** Se desarrolló un transformador compatible con la API de Scikit-learn. Este componente recibe un DataFrame de Pandas con las columnas `title` y `text`, las concatena y aplica la función `preprocess_text` (definida en `core/config.py`). Dicha función ejecuta una secuencia estandarizada de limpieza: conversión a minúsculas, eliminación de signos de puntuación y dígitos numéricos, tokenización por espacios, eliminación de stopwords y reducción de palabras a su raíz mediante stemming.
2. **Vectorización TF-IDF:** El texto preprocesado es transformado en una matriz numérica de características utilizando el esquema Term Frequency-Inverse Document Frequency. Se estableció un límite de 5000 características para controlar la dimensionalidad y optimizar el rendimiento, basado en análisis exploratorios previos.
3. **Clasificación:** Se instanció el modelo Random Forest, seleccionado como el de mejor desempeño general en la Etapa 1 (fue documentado en el anterior documento). Se utilizaron los hiperparámetros usados durante dicha etapa para replicabilidad, además se usó la bibliografía descrita en la Wiki del proyecto para escoger los mejores parámetros.

¹URL Repositorio: <https://github.com/GROUP-BI/Proyecto1.git>

La persistencia del pipeline se realiza utilizando la función `dump` de la biblioteca. El objeto serializado se almacena en la ruta definida por la variable de entorno `MODEL_PATH` (por defecto, `models/pipeline_model.joblib`). Las funciones auxiliares `save_pipeline` y `load_pipeline` en `pipeline/pipeline.py` gestionan de forma segura estas operaciones de guardado y carga, incluyendo manejo de errores básicos como `FileNotFoundError`.

2.2 Desarrollo de la API REST con FastAPI

Para desacoplar el modelo de la aplicación frontend y permitir su consumo por diversos clientes potenciales, se desarrolló una API REST. Se seleccionó el framework FastAPI debido a sus ventajas en cuanto a rendimiento, facilidad para definir endpoints con tipos de datos Python, validación automática de solicitudes y respuestas.

La API, cuya implementación principal reside en `main.py` y el directorio `api/`, expone los siguientes endpoints bajo el prefijo `/api/v1`, siguiendo los requisitos funcionales del proyecto:

1. Endpoint de Predicción:

- ◇ *Funcionalidad:* Permite obtener predicciones para una o más noticias.
- ◇ *Entrada:* Recibe en el cuerpo de la solicitud un objeto JSON conforme al schema `PredictionRequest`, que contiene una lista de objetos `NewsInput`.
- ◇ *Procesamiento :* Valida la solicitud con Pydantic. Carga el pipeline `.joblib` mediante `load_pipeline`. Transforma la lista de entrada en un `DataFrame` de Pandas. Ejecuta los métodos `predict()` y `predict_proba()` del pipeline sobre el `DataFrame`. Adicionalmente, invoca la función `extract_keywords_from_t` que reutiliza el vectorizador TF-IDF ajustado del pipeline para extraer las N palabras clave con mayor puntaje TF-IDF para cada texto de entrada.
- ◇ *Salida:* Devuelve una respuesta JSON con código 200 OK. Contiene una lista de objetos `PredictionOutput`, manteniendo el orden y cardinalidad de la entrada. Cada `PredictionOutput` incluye el id original, la `prediction` textual, la `probability` de esa predicción, y la lista de `keywords` extraídas. También retorna un identificador de la `model_version`.

2. Endpoint de Reentrenamiento:

- ◇ *Funcionalidad:* Permite actualizar el modelo utilizando un nuevo conjunto de datos etiquetados.
- ◇ *Entrada:* Recibe en el cuerpo de la solicitud un objeto JSON conforme al schema `RetrainRequest`, que contiene una lista de objetos `RetrainInput`.
- ◇ *Procesamiento:* Valida la entrada. Utiliza el sistema `BackgroundTasks` de FastAPI para ejecutar el proceso de reentrenamiento de forma asíncrona, evitando bloqueos largos en la respuesta. La tarea en segundo plano convierte los datos a un `DataFrame`, invoca `retrain_pipeline_with_new_data` para entrenar un nuevo pipeline con estos datos, calcula métricas de desempeño sobre los mismos datos de reentrenamiento, asigna un timestamp como versión al nuevo pipeline, lo guarda sobrescribiendo el archivo `.joblib` anterior y registra las métricas, aso como el timestamp en el log del servidor.
- ◇ *Salida:* Devuelve una respuesta inmediata JSON con código 202 Accepted, confirmando el inicio del proceso. La respuesta no incluye las métricas, dado que se calculan asíncronamente.

2.3 Estrategias de Reentrenamiento Implementada

Se investigaron tres enfoques principalmente:

1. **Reentrenamiento Completo (Nuevos Datos):** Entrenar un modelo nuevo usando solo el último lote de datos disponibles. *Ventaja:* Simple, rápido si los datos son pocos, adaptativo a cambios recientes. *Desventaja:* Olvido, dependencia de la representatividad del nuevo lote.

2. **Reentrenamiento Completo (Acumulativo):** Entrenar un modelo nuevo usando la unión de los datos históricos y el nuevo lote. *Ventaja:* Retiene conocimiento histórico, potencialmente más robusto. *Desventaja:* Costo computacional y de almacenamiento creciente.
3. **Aprendizaje Incremental:** Actualizar el modelo existente con los nuevos datos sin reentrenar desde cero. *Ventaja:* Eficiente en recursos, rápido para flujos continuos. *Desventaja:* No universalmente aplicable.

Estrategia Implementada: Para este proyecto, se optó por implementar la estrategia de Reentrenamiento (sobre Nuevos Datos), es decir la `opc1`, como se refleja en la lógica de la función `retrain_pipeline_with_new_data`. *Justificación de la Elección:* Esta decisión se basó en un balance entre simplicidad de implementación dentro del alcance del proyecto y la necesidad de adaptación a la naturaleza potencialmente cambiante de las tácticas de desinformación. Se asumió que los datos más recientes podrían ser más indicativos de las tendencias actuales, con ello se brinda un mejor contexto.

3 Desarrollo de la Aplicación y Justificación

Más allá de la automatización del modelo, un objetivo clave de la Etapa 2 fue el desarrollo de una aplicación frontend que permite a un usuario final interactuar de manera significativa con la solución analítica.

3.1 Descripción del Usuario/Rol y Proceso de Negocio

Basado en el análisis realizado en la Etapa 1 y plasmado en el Canvas CRISP-ML(Q), la aplicación "Fake News Detector" se diseñó teniendo como usuario final principal a un equipo de relaciones públicas o comunicación estratégica dentro de una organización gubernamental en Colombia.

La importancia de esta aplicación para dicho rol es multifacética:

- ◊ Permite un monitoreo proactivo y una detección temprana de noticias potencialmente falsas que podrían afectar la reputación de la organización o la percepción pública sobre temas críticos.
- ◊ Facilita una respuesta más ágil y basada en evidencia ante posibles crisis de desinformación, al proporcionar una evaluación inicial rápida sobre la veracidad de un contenido.
- ◊ Ofrece insights que pueden ayudar a comprender las narrativas de desinformación circulantes y ajustar las estrategias de comunicación.

La aplicación se integra en el flujo de trabajo del equipo de comunicaciones como una herramienta de apoyo muy fuerte:

1. Ante la aparición de una noticia de alto impacto o dudosa procedencia en medios digitales o redes sociales, un miembro del equipo utiliza la pestaña "Single Analysis" para obtener una evaluación inicial rápida.
2. El resultado informa la decisión sobre si escalar la noticia para una verificación humana profunda o si descartarla preliminarmente.
3. Periódicamente, el equipo puede usar "Batch Analysis" para analizar un conjunto de noticias recopiladas de monitores de medios o revisar Insights: "History" para identificar tendencias y patrones de desinformación relevantes para la organización.
4. En casos donde se confirma fehacientemente la falsedad o veracidad de noticias analizadas, un usuario experto podría utilizar la funcionalidad de "Model Training" para contribuir a la mejora continua del modelo con datos verificados.

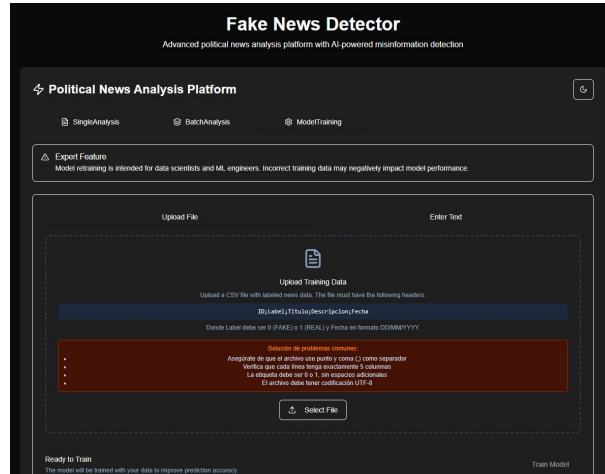


Figura 1: Interfaz de la aplicación

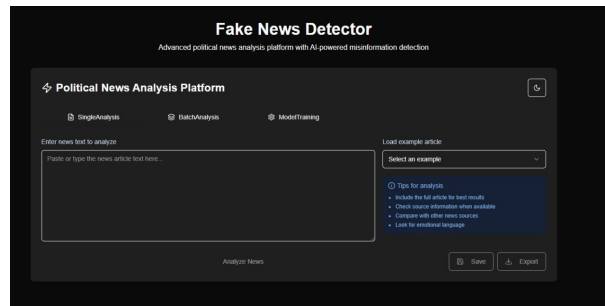


Figura 2: Interfaz de la aplicación 2

3.2 Validación del Diseño con el Usuario

El diseño y desarrollo de la aplicación "Fake News Detector" se guiaron por un enfoque en el usuario final. Basándose en el análisis de la Etapa 1 y la definición del beneficiario principal (equipo de comunicación/relaciones públicas), el equipo elaboró perfiles de usuario que representaban las necesidades, tareas y contexto de uso de estos profesionales.

Estos perfiles sirvieron como referencia constante durante el diseño de la interfaz y la definición de las funcionalidades. La validación de que estos perfiles eran acertados y que la aplicación respondía efectivamente a las necesidades identificadas no se realizó con un grupo formal externo específico, sino que se integró directamente en el proceso de pruebas de usabilidad y utilidad descrito en la Sección 4.4. Durante dichas pruebas (detalladas más adelante), la interacción directa con usuarios potenciales permitió confirmar la relevancia de las funcionalidades ofrecidas para los roles definidos y recoger retroalimentación valiosa que informó ajustes finales en la interfaz y la presentación de la información.

3.3 Análisis Técnico: Recursos, Integración y Riesgos

Se realizó un análisis de los requisitos técnicos y riesgos asociados al despliegue de la aplicación:

- ◊ **Recursos Informáticos Requeridos:** El backend requiere un entorno Python con las librerías listadas en `requirements.txt`. Puede operar en un contenedor Docker para facilitar el despliegue. El consumo de RAM está dominado por la carga del modelo `.joblib`. El entrenamiento inicial o reentrenamientos completos con datasets grandes requerirían recursos más significativos (CPU, RAM) temporalmente.

- ◊ **Integración y Puesta a Disposición:** La aplicación está diseñada como una herramienta web autónoma. Se pondría a disposición de los usuarios mediante una URL accesible. No se prevén integraciones directas con otros sistemas legados en esta fase, aunque la API permitiría futuras integraciones.
- ◊ **Riesgos para el Usuario Final:** Además de los mencionados en la Etapa 1 (mala interpretación, sesgos, falsos positivos/negativos), se considera:
 - ◊ *Dependencia y Automatización Excesiva:* El usuario podría volverse excesivamente dependiente de la herramienta, reduciendo su propio juicio crítico o habilidades de verificación manual.
 - ◊ *Seguridad de Datos:* Si la funcionalidad de reentrenamiento maneja datos sensibles, se deben implementar medidas de seguridad robustas (autenticación, autorización, protección de datos en tránsito y reposo) que exceden el alcance actual del proyecto.
 - ◊ *Mantenimiento del Modelo:* Un modelo no actualizado puede degradar su rendimiento. El proceso de reentrenamiento debe ser regular y supervisado.

4 Resultados

Esta sección muestra los resultados de la Etapa 2, enfocándose en la demostración funcional de la aplicación y la evaluación de su recepción por parte de usuarios potenciales.

4.1 Video Demostrativo y Visualización

Se produjo un video que demuestra de manera práctica la aplicación "Fake News Detector". El video se encuentra disponible públicamente en el enlace de la Wiki.

4.2 Acciones Habilitadas por la Aplicación

El video y el diseño de la aplicación destacan cómo la herramienta habilita al usuario final (equipo de comunicación/relaciones públicas) para realizar acciones informadas y estratégicas. Las dos acciones principales demostradas son:

1. Acción 1: Priorización Eficiente de Verificación Manual.

- ◊ *Descripción:* Ante un volumen elevado de noticias o alertas de monitoreo, el usuario introduce el texto de las noticias sospechosas en la aplicación. Utiliza la predicción y, crucialmente, el score de probabilidad para filtrar y priorizar. Noticias clasificadas como 'FAKE' con una alta probabilidad son inmediatamente escaladas al equipo de fact-checking o para una investigación interna profunda. Noticias clasificadas como 'REAL' con alta probabilidad o 'FAKE' con baja probabilidad pueden ser descartadas o puestas en una cola de menor prioridad.
- ◊ *Aporte del Modelo:* El modelo actúa como un primer filtro inteligente, permitiendo al equipo enfocar sus recursos limitados de verificación humana en los casos de mayor riesgo o probabilidad de ser desinformación dañina. Reduce significativamente el tiempo dedicado a revisar contenido irrelevante o claramente verídico. Las keywords ofrecen pistas iniciales para la verificación.

2. Acción 2: Identificación Temprana de Campañas o Narrativas.

- ◊ *Descripción:* Utilizando la funcionalidad de "Batch Analysis", el usuario puede detectar patrones. Por ejemplo, si múltiples noticias clasificadas como 'FAKE' en un corto período comparten palabras clave específicas o temas similares, esto puede ser un indicador temprano de una campaña de desinformación coordinada sobre un asunto particular.

- ◊ *Aporte del Modelo:* Más allá del análisis individual, el modelo, al ser aplicado sistemáticamente, permite identificar tendencias. Las keywords extraídas y los análisis agregados ayudan a caracterizar estas campañas, permitiendo al equipo de comunicación preparar contra-narrativas o alertas preventivas de manera más informada y proactiva.

4.3 Reflexión sobre el Impacto Potencial en Colombia

La implementación de herramientas como "Fake News Detector" en el contexto colombiano da un potencial considerable, aunque no exento de desafíos. Por un lado, podría agilizar significativamente el trabajo de periodistas, verificadores y equipos de comunicación institucionales, permitiéndoles responder más rápidamente a la desinformación y contribuyendo a un debate público más informado, especialmente en periodos electorales o de alta polarización. Podría, en teoría, aumentar la alfabetización mediática si sus resultados se comunican de forma responsable al público general. Sin embargo, los riesgos son igualmente importantes. La dependencia excesiva de la herramienta sin verificación humana podría perpetuar errores o sesgos del modelo. Existe el riesgo de que actores malintencionados utilicen herramientas similares para desacreditar información legítima (falsos positivos). Además, la efectividad del modelo depende de la calidad y representatividad de los datos de entrenamiento, que deben reflejar las particularidades del lenguaje y las tácticas de desinformación usadas en Colombia.

4.4 Pruebas de Facilidad de Uso y Utilidad

Se llevó a cabo un proceso de evaluación centrado en el usuario para valorar la facilidad de uso (usabilidad) y la utilidad percibida de la aplicación "Fake News Detector", cumpliendo así con el requisito de validación con usuarios finales.

Metodología: Se realizaron sesiones de prueba de usabilidad de forma remota con 10 participantes. Estos participantes fueron reclutados por tener perfiles consistentes con el usuario objetivo definido (ej., estudiantes de últimos semestres en áreas como comunicación social, periodismo, ciencias políticas o marketing digital). A cada participante se le presentó un breve contexto de la aplicación y se le solicitaron realizar un conjunto de tareas representativas:

- ◊ Analizar una noticia proporcionada usando la función "Single Analysis".
- ◊ Interpretar los resultados mostrados (predicción, probabilidad, palabras clave).
- ◊ Localizar y describir el propósito de la función de "Model Training".

Durante la ejecución de las tareas, se aplicó la técnica de "pensamiento en voz alta" para capturar impresiones y dificultades en tiempo real. Al finalizar la sesión, recibimos su retroalimentación de manera informal.

Resultados Principales: El análisis cualitativo del feedback obtenido mediante "think-aloud" corroboró una percepción positiva. Los participantes elogiaron mayoritariamente la limpieza de la interfaz y la claridad con la que se presentan los resultados del análisis principal. La utilidad de la herramienta para obtener una evaluación rápida inicial fue un punto recurrente de valoración positiva. No obstante, se identificaron también áreas de mejora consistentes:

- ◊ Sugerencias para hacer más intuitivo el proceso de carga de archivos en la funcionalidad "Batch Analysis".
- ◊ Algunos participantes expresaron la necesidad de tooltips o ayudas contextuales para entender mejor la métrica de probabilidad.

Estos resultados validan que la aplicación es percibida como usable y útil por su público objetivo, al tiempo que proporcionan insumos valiosos para futuras iteraciones de mejora.

5 Trabajo en Equipo

La de la Etapa 2 requirió una coordinación y colaboración efectivas entre los miembros del equipo. Esta sección detalla la organización interna, la distribución del trabajo, los desafíos superados y la evaluación colaborativa.

5.1 Roles, Tareas Realizadas y Tiempos Dedicados

El equipo adoptó una estructura de roles, inspirada en las sugerencias de la guía del proyecto, para abordar las diversas responsabilidades de ingeniería de datos y software. La Tabla 1 resume las contribuciones individuales:

Cuadro 1: Distribución de Roles, Tareas y Dedicación Horaria (Etapa 2)

Integrante	Rol(es) Principal(es)	Tareas Específicas Realizadas	Horas (Aprox.)
Andres Mateo Chilito Avella	Líder Proyecto, Ing. Datos	Gestión general, planificación, automatización pipeline (core), desarrollo endpoint <code>/predict</code> , documentación API, coordinación GAEA.	[RV: 45 hrs]
Franklin Smith Fernandez Romero	Ing. Software (Diseño/-Resultados), Ing. Datos	Diseño UI/UX frontend, desarrollo endpoint <code>/retrain</code> , implementación lógica background tasks, creación video, pruebas usabilidad, documentación resultados.	[RV: 50 hrs]
Juan Jose Cediel Borrero	Ing. Software	Implementación frontend , conexión con API, gestión estado frontend, configuración inicial despliegue, documentación técnica código frontend.	[RV: 40 hrs]

5.2 Retos Enfrentados y Soluciones Planteadas

El desarrollo de esta etapa presentó desafíos técnicos y organizativos que requirieron soluciones adaptativas:

◊ **Reto 1: Gestión del Estado Asíncrono del Reentrenamiento.**

- *Descripción:* Asegurar que, tras completarse una tarea de reentrenamiento en segundo plano, las nuevas solicitudes al endpoint `/predict` utilizaran efectivamente el modelo recién actualizado, sin requerir un reinicio manual de la API.
- *Solución:* Se implementó un mecanismo donde la función `load_pipeline` no solo carga desde el archivo si la caché en memoria es nula, sino que también verifica periódicamente o bajo demanda si el archivo `.joblib` ha sido modificado (comparando timestamps de archivo/memoria) para recargar el modelo actualizado de forma transparente.

◊ **Reto 2: Alineación Interdisciplinaria con GAEA.**

- *Descripción:* Traducir los requisitos y el feedback del grupo de estadística (con un enfoque más analítico y metodológico) en especificaciones técnicas claras y accionables para el desarrollo de la interfaz y la presentación de resultados.

- *Solución:* Se adoptó un enfoque de comunicación iterativa, utilizando prototipos visuales (mockups, demos tempranas) para facilitar la discusión. Se designó a un miembro del equipo como enlace principal para consolidar y traducir la retroalimentación, asegurando que las decisiones de diseño tuvieran una justificación clara basada en las necesidades validadas.

5.3 Distribución de Puntos y Puntos a Mejorar

En un ejercicio de autoevaluación y consenso, considerando la dedicación, calidad, cumplimiento y equidad en las contribuciones individuales durante la Etapa 2, el equipo acordó la siguiente distribución de los 100 puntos asignados al trabajo en equipo:

Cuadro 2: Distribución de Puntos de Trabajo en Equipo (Etapa 2)

Integrante	Puntos Asignados (sobre 100)	Justificación Breve
Andres Mateo Chilito Avella	[RV: 33.3]	Liderazgo proactivo, responsabilidad clave en backend y pipeline, coordinación efectiva con GAEA.
Franklin Smith Fernandez Romero	[RV: 33.3]	Contribución fundamental en diseño UI/UX, endpoint de reentrenamiento, video y pruebas. Alta calidad en entregables visuales.
Juan Jose Cediel Borrero	[RV: 33.3]	Desarrollo sólido del frontend, implementación efectiva de la conexión API, buena disposición para tareas técnicas asignadas.
Total	100	

Como reflexión final, el equipo identificó las siguientes áreas clave para mejorar en futuros proyectos:

- ◊ **Planificación Técnica Detallada:** Dedicar más tiempo al inicio para definir interfaces internas (entre frontend y backend, entre módulos del backend) de forma más precisa para minimizar retrabajos de integración.
- ◊ **Gestión de Dependencias:** Utilizar herramientas de gestión de entornos virtuales de manera más rigurosa para asegurar la reproducibilidad exacta del entorno de desarrollo y producción.

6 Conclusiones

La culminación de la Etapa 2 del proyecto de detección de noticias falsas representa un avance significativo desde la modelación analítica hacia una solución de software funcional y potencialmente desplegable. Se logró con éxito la automatización del pipeline de preprocesamiento, entrenamiento y evaluación del modelo Random Forest mediante Scikit-learn y 'joblib', asegurando la reproducibilidad y eficiencia del flujo de trabajo.

La implementación de una API REST utilizando FastAPI proporciona una interfaz robusta y estándar para interactuar con el modelo, ofreciendo endpoints para predicción y reentrenamiento asíncrono, elementos cruciales para la mantenibilidad y extensibilidad de la solución.

El desarrollo de la aplicación web "Fake News Detector" materializa la utilidad del modelo para un usuario final definido (equipo de comunicación/relaciones públicas). La interfaz permite una interacción intuitiva

para analizar noticias, visualizar resultados y, para usuarios expertos, gestionar el reentrenamiento del modelo.

Si bien existen desafíos inherentes y áreas de mejora continua (como el manejo de sesgos, estrategias de reentrenamiento más avanzadas y robustez a escala), este proyecto sienta las bases para una herramienta valiosa en el combate contra la desinformación. El código fuente completo, la documentación detallada, el video demostrativo y los resultados de las pruebas se encuentran disponibles en la Wiki del repositorio del proyecto.

Referencias

- L. D. Pinos-Velez, V. H. Andaluz, and M. E. Torres, “Detección automática de noticias falsas en español: Sátira política y noticias reales,” *Revista Politécnica*, vol. 44, no. 2, pp. 7–16, Dec. 2019, doi: 10.33333/rp.vol44n2.1355. [Online]. Available: https://revistapolitecnica.epn.edu.ec/ojs2/index.php/revista_politecnica2/article/view/1355
- S. A. Hasan, M. A. Shajal, F. M. Al-khateeb, and R. A. Koteb, “Fake news detection in Spanish using deep learning techniques,” arXiv preprint arXiv:2110.06461, 2021. [Online]. Available: <https://arxiv.org/abs/2110.06461>
- G. I. Fitt, “Aplicación de técnicas de aprendizaje automático para la clasificación temática de textos académicos,” Undergraduate Thesis, Fac. de Ciencias Exactas, Ingeniería y Agrimensura, Univ. Nacional de Rosario, Rosario, Argentina, 2018. [Online]. Available: <https://rehip.unr.edu.ar/handle/2133/12515>